MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Governance Framework and an OWL 2 Ontology for the Common Provenance Framework

Master's Thesis

STANISLAV PŘIKRYL

Brno, Spring 2026

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Governance Framework and an OWL 2 Ontology for the Common Provenance Framework

Master's Thesis

STANISLAV PŘIKRYL

Advisor: RNDr. Rudolf Wittner

Department of Machine Learning and Data Processing

Brno, Spring 2026


FACULTAS ARTIS INFORMATICÆ
UNIVERSITAS MASARYKIANA

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Stanislav Přikryl

Advisor: RNDr. Rudolf Wittner

Acknowledgements

These are the acknowledgements for my thesis, which can span multiple paragraphs.

Abstract

This is the abstract of my thesis, which can
span multiple paragraphs.

Keywords

keyword1, keyword2, …

Contents

List of Tables

List of Figures

1 Introduction

Here begins the introduction of the thesis...

2 Background and State of the Art

This chapter establishes the theoretical and technological foundation of the thesis. It introduces the core concepts of data provenance and reviews existing international standards, with a primary focus on the W3C PROV-O ontology, which provides the baseline vocabulary for provenance representation. Furthermore, it examines the RO-Crate specification as a modern paradigm for packaging research data. Finally, the chapter provides a comprehensive summary of the Common Provenance Framework (CPF) specification, establishing the baseline requirements and multi-organizational challenges that the subsequent governance model and technical artifacts aim to address.

3 Design of the Governance Framework

To ensure the systematic evolution, maintainability, and interoperability of the Common Provenance Framework (CPF) across distributed environments, its architecture and semantics are formalized into a strict governance model. This framework is conceptually divided into nine interconnected modules (Module 0 to Module 8). Each module has a strictly defined scope of responsibility, managed concepts, and a change decision matrix.

3.1 Module 0: Terminology

The fundamental prerequisite for any robust distributed ecosystem is a shared, unambiguous vocabulary. Within the **Design Science Research Process (DSRP)**, Module 0 represents the transition from problem identification to the definition of objectives, establishing the semantic constraints that the artifact (the Governance Framework) must uphold. Module 0 serves as the normative foundation of the Common Provenance Framework (CPF), ensuring alignment with the **ISO 23494** series while providing the specific extensions required for multi-organizational provenance.

In large-scale environments, there is a significant risk of *semantic drift*. For instance, the term "Agent" could be interpreted as a legal entity in a regulatory context, or a software daemon in a technical one. Module 0 mitigates this by defining semantics in natural language, decoupled from technical implementation, yet strictly mapped to the **Common Provenance Model (CPM)**.

3.1.1 Core Responsibilities and Managed Concepts

The primary responsibility of this module is to ensure *semantic interoperability*. Manages the definitions of both abstract domain concepts and structural elements as defined in the CPF Technical Supplementary:

- **Provenance Information vs. Provenance Component:** A critical distinction governed by this module is between raw *Provenance Information* (the records of a resource's history) and a

Provenance Component (the manageable, finalized unit of provenance data encapsulated in a PROV bundle).

- **Core PROV-DM Elements:** Module 0 adopts and constrains the three pillars of the W3C PROV-DM, specifically:
  - *Entities:* Passive objects, encompassing both physical samples (e.g., biological specimens) and digital data (e.g., FASTQ files).
  - *Activities:* Dynamic processes that utilize or produce entities.
  - *Agents:* Entities bearing responsibility for activities. In the CPF context, this module explicitly defines the roles of **Controller, Processor, and Provider**, ensuring legal and technical accountability is distinguishable.
- **Connectors (Topological Elements):** This module governs the taxonomy of *Forward* and *Backward Connectors*. It defines them as specialized entities used to maintain the integrity of the provenance chain across organizational boundaries, even when specific segments of the chain are inaccessible.
- **Described Object:** A unique CPM concept referring to the specific entity whose provenance is being tracked. Module 0 defines the criteria for what constitutes a valid described object (e.g., traceability requirements).

3.1.2 Scope Boundaries (Out of Scope)

To maintain a clean separation of concerns (Separation of Concerns), Module 0 excludes:

1. **Formal Syntax:** While this module defines what a "Forward Connector" is conceptually, the formal RDF mapping and property constraints (e.g., `cpm:referencedBundleId`) are strictly managed by **Module 9 (CPM Ontology)**.
2. **Procedural Logic:** The rules for how to traverse these connectors to reconstruct a history are relegated to **Module 3 (Traversal Mechanics)** and **Module 6 (Algorithms)**.

3.1.3 Change Decision Matrix (Governance Logic)

As an element of the DSRP artifact, any modification to Module 0 triggers a high-impact change protocol due to the cascading nature of terminology:

- **Semantic Expansion (Minor/Patch):** Clarifying a definition without altering its logical constraints (e.g., refining the description of a Processor" role) results in a patch.

- **Structural Redefinition (Major):** If a core concept is redefined (e.g., changing the role of a Connector from an Entity to an Activity), it constitutes a breaking change. This requires a mandatory update of all downstream modules, particularly the SHACL shapes in Module 9.

- **Domain Isolation (Rejection):** Proposed terms specific to a single domain (e.g., Genomic Variant") are rejected from the Core Terminology. The CPF governance dictates these must be managed in **Domain Extensions**, keeping the core framework domain-agnostic.

3.1.4 Edge Cases and Boundary Resolution

A key edge case handled by this module is the **representation of physical-to-digital transitions**. For example, when a tissue sample (Physical Entity) is sequenced into data (Digital Entity), is the resulting data a new "Described Object" or a continuation of the old one?

According to CPF governance, Module 0 provides the resolution: a transition in the nature of the entity (physical to digital) constitutes a transformation activity. The terminology module mandates that in such cases, the CPM must support *multi-layered described objects*, where the digital entity maintains a specialized link to its physical ancestor, governed by the sémantice of the "Specialized Connector" defined here.

3.2 Module 1: Architectural Requirements and Constraints

While Module 0 establishes the vocabulary, Module 1 defines the fundamental technical invariants and requirements (R1–R11) of the Common Provenance Framework. In the context of the **Design Science Research Process (DSRP)**, this module specifies the *Objectives for a Solution*, acting as the qualitative and quantitative "constitution" that the resulting artifact must satisfy. For the framework to guarantee interoperability, trustworthiness, and data integrity in a distributed environment, these constraints must be universally respected.

3.2.1 Core Responsibilities and Managed Requirements

Module 1 manages the interpretation and enforcement of rules across several critical domains, as dictated by the core CPF specification:

- **Decentralization and Sovereignty (R1, R10):** The framework strictly prohibits reliance on any central authority or centralized storage registry. It mandates that provenance generation remain localized at the origin (Sovereignty). The governance model ensures that no single point of failure exists and that organizations maintain control over their internal data representation.
- **Resilience and Integrity (R5, R7):** The architecture must support *disconnected operations*, allowing nodes to record provenance data even without active connectivity. Crucially, it mandates **Redundancy of Links** (e.g., via redundant connectors) to ensure that the provenance chain remains traversable even if intermediate nodes or their services become permanently unavailable.
- **Privacy and Granularity (R3, R6):** Module 1 dictates that the framework must support varying levels of data abstraction. It enforces a "Need-to-Know" principle, where the technical structure (CPM) must allow for the redaction or obfuscation of sensitive information without breaking the cryptographic integrity of the overall chain.

- **Non-Repudiation of Origin (NRO):** This module establishes the security invariants based on the **NRO-P1 to NRO-P10** policies. It mandates an unbreakable cryptographic binding between the provenance bundle (FPC), the identity of its originator, and the exact timestamp, ensuring that the creation of provenance cannot be successfully denied later.

3.2.2 Scope Boundaries (Out of Scope)

To prevent architectural bloat, Module 1 focuses solely on systemic properties. It does not:

1. Define **Functional Implementations:** While it mandates that a cryptographic signature *must* exist to satisfy NRO-P1, it does not dictate the specific implementation (e.g., RSA vs. ECDSA); that is deferred to **Module 5 (Security)**.
2. Manage **Data Schemas:** The specific RDF properties and OWL classes used to represent these constraints are the jurisdiction of **Module 9 (CPM Ontology)**.

3.2.3 Change Decision Matrix

Because this module governs the fundamental properties of the CPF, modifications are subjected to the strictest evaluation to prevent "Requirement Drift":

- **Paradigm Shift (Major):** Any proposal that introduces a centralized component (e.g., a global ID registry) violates R10 and constitutes a breaking change, requiring a new major version of the framework.
- **Requirement Relaxation (Major):** Proposing to make R5 (Resilience) optional for specific implementations is considered a structural violation of the CPF core and is generally rejected or requires a fork status.
- **Technical Specification (Rejection/Delegation):** A proposal specifying that an identifier must follow a specific URI format (e.g., DOI vs. UUID) does not alter the fundamental constraints; it is therefore delegated to **Module 2 (Identity & Linkage)**.

3.2.4 Boundary Resolution and Edge Cases

A critical edge case in multi-organizational governance is the **"Trust vs. Compliance" dilemma**. For instance, a consortium of highly trusted research hospitals may propose a "Trusted-Circle" implementation where NRO signatures are omitted to reduce latency.

Under the CPF governance model, this is resolved via the **Compliance Invariant**. The decision dictates that while trust may exist at the organizational level, it cannot substitute for technical evidence. Bypassing NRO is a direct violation of Module 1. The governance body may certify such an implementation as *"CPF-Compatible"* for internal use, but never as *"CPF-Compliant"*. This ensures that if a sample is later shared with an external party outside the "circle," the provenance integrity remains intact and verifiable by the global ecosystem, upholding the R4 (Interoperability) requirement.

3.3 Module 2: CPM Identity & Linkage

In a fully distributed ecosystem lacking a central authority, uniquely identifying and cross-referencing data across independent systems represents a critical architectural challenge. Within the **Design Science Research Process (DSRP)**, Module 2 specifies the structural design of the artifact's identification mechanisms. It governs the rules for identity generation, structural representation, and the distributed linkage patterns necessary to satisfy the interoperability requirement (R4) defined in Module 1.

3.3.1 Core Responsibilities and Managed Concepts

Rather than defining the topological placement of an identifier within the graph, this module strictly governs its data representation and generation mechanics. It manages the following core concepts:

- **Global URI Formulation:** The module dictates the mandatory informational components required for global uniqueness of nodes (Agents, Activities, Entities) and Finalized Provenance Components (FPCs). It mandates the use of resolvable URIs that combine a namespace/authority with a local identifier.

- **External Identity Mapping:** A unique responsibility of this module is governing the bridge between the physical world and the digital graph. It controls the structures representing legacy or real-world identifiers (e.g., hospital barcodes) using mandatory tuple structures, specifically the combination of `cpm:externalId` and `cpm:externalIdType`.

- **Cryptographic Linkage Verification:** To maintain trust across distributed components, this module defines the structural schema for linkage. It mandates that any reference to an external provenance bundle (e.g., via a Forward or Backward Connector) must include Verification Data. This is structurally represented by `cpm:hashValue` and `cpm:hashAlg`, ensuring the referenced identity remains tamper-evident.

3.3.2 Scope Boundaries (Out of Scope)

A strict boundary is drawn between identity logic, graph topology, and ontological syntax:

1. **Topological Cardinality:** Module 2 defines the structure of the "identity card," but it does not dictate who must hold it. For example, the rule stating that a *Sender Agent* node *must* possess an external identifier belongs strictly to **Module 3 (Traversal Mechanics)**.

2. **Syntactic Validation:** While this module defines the conceptual requirement for a tuple (ID + Type), the strict enforcement via SHACL shapes is delegated to **Module 9 (CPM Ontology)**.

3. **Non-Repudiation Signatures:** Module 2 governs the hashing mechanism used to identify and link content, but it does not govern the cryptographic signatures used for Non-Repudiation of Origin (NRO). That process is exclusively managed by **Module 5 (Security)**.

3.3.3 Change Decision Matrix

Changes to identity construction can severely impact the ability to traverse historical data (backward compatibility). Proposals are evaluated as follows:

- **Fundamental Format Alteration (Major Change):** Shifting the core logic of uniqueness (e.g., abandoning URIs for distributed identifiers like DIDs without backward compatibility) represents a breaking change. This requires a Major version update and forces cascading updates to **Module 8 (Reference Software)** parsers.
- **Algorithm Expansion (Minor Change):** Adding support for a new hashing algorithm (e.g., introducing SHA-3 alongside existing SHA-256) extends the specification without breaking existing data linkages, qualifying as a Minor change.
- **Namespace Deprecation (Patch):** Updating the documentation to reflect a retired institutional namespace, without altering the parsing logic, is a Patch.

3.3.4 Boundary Resolution and Edge Cases

A critical edge case in multi-organizational life sciences is the **External Identifier Collision**. For instance, Hospital A and Hospital B might both assign the local barcode "SMPL-001" to two completely different physical tissue samples. A developer might propose simplifying the framework by dropping the `cpm:externalIdType` attribute to reduce payload size, assuming the URI of the institution provides enough context.

Under the CPF governance framework, this proposal triggers a structural validation against Module 2. The decision dictates that an external identifier is conceptually incomplete without its typing context. Module 2 strictly mandates the tuple (`cpm:externalId` + `cpm:externalIdType`) to guarantee deterministic resolution of real-world objects. The proposal is rejected. If a genuine need for payload reduction exists, it must be solved via compression in **Module 7 (Exchange Protocols)**, not by compromising the structural integrity of identity governed by Module 2.

3.4 Module 3: CPM Traversal Backbone

Derived from the *Traversal Backbone Pattern* introduced in the core CPF specification, this module serves as the topological heart of the entire framework. Within the **Design Science Research Process (DSRP)**, this module dictates the structural architecture of the artifact, defining the valid logical pathways through the distributed data. It ensures that the resulting provenance network is continuous, causally sound, and fully traversable across organizational boundaries.

Crucially, Module 3 guarantees domain independence. It abstracts away domain-specific processes, providing only the universal building blocks necessary for constructing a distributed chain.

3.4.1 Core Responsibilities and Managed Concepts

This module manages the fundamental taxonomy and relational rules of the graph structure. It explicitly extends the W3C PROV-DM standard with CPF-specific topological constraints:

- **CPF Node Taxonomy:** While recognizing the base PROV-DM entities (Entities, Activities, Agents), Module 3 governs the specialized CPM topological roles. It strictly defines the usage of `cpm:MainActivity` (the core process within a bundle), and the precise multi-organizational agent roles: `cpm:SenderAgent`, `cpm:ReceiverAgent`, and `cpm:CurrentAgent`.

- **Directional Connectors:** The formal definition of edges essential for inter-bundle traversal. This includes *Backward Connectors* (linking to historical causes), *Forward Connectors* (linking to subsequent effects), and *Forward Connector Specialized* (specifically governed rules for tracking the transfer of physical material, requiring contact PIDs).

- **Linkage Attributes:** The exact definition of mandatory properties required to bridge isolated components. Module 3 dictates that a valid topological link must contain references not just to the data payload (`cpm:referencedBundleId`), but crucially to its governance layer (`cpm:referencedMetaBundleId`) to maintain version awareness during traversal.

- **Topological Rules and Cardinalities:** Structural invariants that dictate graph continuity. For example, the rule that a `cpm:SenderAgent` in one bundle logically corresponds to a `cpm:ReceiverAgent` in the subsequent bundle.

3.4.2 Scope Boundaries (Out of Scope)

To maintain a strict Separation of Concerns, Module 3 delegates several aspects to other modules:

- **Identifier Formats:** While the Backbone mandates the *presence* of a reference identifier to form a link, the internal parsing logic and validation of that URI/Hash is exclusively managed by **Module 2 (Identity & Linkage)**.
- **Execution Algorithms:** Module 3 defines the "road network" (the valid pathways), but the concrete execution logic (e.g., how a `traceForward()` recursive function evaluates these nodes) belongs to **Module 6 (Traversal Algorithm)**.
- **Syntactic Enforcement:** The natural language rules defined here are translated into machine-actionable validation constraints by **Module 9 (CPM Ontology & SHACL)**.

3.4.3 Change Decision Matrix

Modifications to the backbone directly affect the topology of the provenance graph, potentially breaking historical traversability. They are evaluated rigorously:

- **Topological Alteration (Major Change):** Introducing a new foundational connector type or altering the arity of existing connections (e.g., allowing a Forward Connector to point to multiple independent bundles simultaneously) represents a major structural shift. This requires a Major version update and direct propagation to Modules 6 and 9.
- **Navigational Extension (Minor Change):** Adding a new optional attribute to a connector to assist in traversal efficiency

(such as a `hopCount` to limit query depth) extends the framework without breaking existing graphs, constituting a Minor change.

3.4.4 Boundary Resolution and Edge Cases

A highly instructive edge case governed by this module is the resolution of **Process Fragmentation (Opaque vs. Transparent Subgraphs)**. A hospital might use 50 internal steps to sequence a genome. Exposing all 50 steps externally would bloat the inter-organizational graph and potentially leak sensitive workflows. The developers propose grouping them into one conceptual "black box."

Under the CPF governance framework, Module 3 provides the exact topological resolution for this scenario. It dictates the implementation of an *Opaque Subgraph*. Module 3 mandates that the internal 50 steps may exist, but the external graph must only see a single `cpm:MainActivity` encapsulating them. If an external auditor needs to trace *into* the hospital's internal steps, Module 3 rules that the `cpm:MainActivity` must act as an expandable node, linking to a secondary, internal provenance bundle. This governance decision ensures that the global traversal algorithm (Module 6) remains fast and uninterrupted, while preserving the depth of local data.

3.5 Module 4: CPM Meta-Bundles, Versioning & Packaging

To ensure seamless portability, interoperability, and lifecycle management across disparate organizations, the CPF employs a standardized container format. Within the **Design Science Research Process (DSRP)**, Module 4 defines the operational encapsulation of the artifact. While Module 3 dictates the abstract causal topology of the graph, Module 4 defines how this graph is physically packaged into a *Finalized Provenance Component (FPC)*, how it is versioned, and how its overarching metadata is structured.

3.5.1 Core Responsibilities and Managed Concepts

The primary responsibility of this module is to govern the *Meta-Bundle* — the informational envelope that wraps the provenance graph. It ensures that every package is self-describing, version-controlled, and ready for integrity validation before the internal graph is parsed.

This module governs the following core concepts:

- **The Meta-Bundle Architecture:** The strict separation between the causal graph (the Bundle) and the administrative data about the graph (the Meta-Bundle). This includes mandatory properties such as `cpm:referencedMetaBundleId` and `cpm:referencedMetaBundleSp` to track the exact specification version.

- **Immutability and Versioning Protocol:** Under CPF rules, a published FPC is strictly immutable. Module 4 dictates the versioning mechanics: any correction or update to historical provenance cannot alter the original file. Instead, a new version must be generated, and the Meta-Bundle is responsible for carrying the *supersedes*/*deprecated* metadata flags.

- **Container Structure (e.g., RO-Crate):** The standardized directory layout of the transport bundle, designating specific locations for the provenance graph (Payload), domain metadata, and digital signatures (Proofs).

3.5.2 Scope Boundaries (Out of Scope)

The governance model establishes a clear boundary using a box and label" analogy. Module 4 manages only the box" (the physical container) and the "label" (the Meta-Bundle and versioning data).

It explicitly excludes:

1. **Internal Graph Logic:** How Entities and Activities are interconnected *inside* the box remains the sole domain of **Module 3 (Backbone)**.

2. **Cryptographic Generation:** While Module 4 dictates the physical location of a signature file (e.g., placing it in a `/proofs/` directory of the RO-Crate), the mathematical algorithms used

to generate that signature are strictly managed by **Module 5 (Security)**.

3.5.3 Change Decision Matrix

Modifications to the packaging and versioning specification directly affect file parsers and network transfer protocols. They are evaluated as follows:

- **Format or Versioning Paradigm Replacement (Major Change):** A proposal to abandon the chosen container standard (e.g., migrating from RO-Crate to a custom ZIP with an XML manifest), or changing how FPC versions are tracked, represents a breaking change. This constitutes a Major version update and forces updates to all client software in **Module 8 (Reference Software)**.
- **Profile Extension (Minor Change):** Adding a new optional or conditionally mandatory field to the Meta-Bundle (such as a specific `license` or `fundingReference` attribute) alters the metadata profile but does not break the underlying parser logic, qualifying as a Minor change.

3.5.4 Boundary Resolution and Edge Cases

A prominent edge case in scientific distributed systems is the **correction of historical "Big Data"**. For example, an institution realizes that a 500 GB genomic dataset, distributed a year ago, contains a calibration error in its provenance graph. The institution needs to update the provenance without re-transmitting the massive 500 GB physical payload, and without violating the immutability constraint (Module 1).

The CPF governance framework resolves this through Module 4's **Detached Payload and Versioning rules**. The resolution dictates that the original FPC (v1) remains cryptographically sealed. The institution must publish a new FPC (v2). Module 4 mandates that this new Meta-Bundle uses a *Detached Architecture*—the container holds only the corrected provenance graph, the new metadata, and cryptographic hashes, while utilizing a URI pointer to reference the original, external

500 GB payload. Furthermore, the v2 Meta-Bundle must explicitly reference v1 as deprecated. This ensures that the framework remains scalable (no massive data duplication) while preserving strict, tamper-evident version history.

3.6 Module 5: CPM Security & Non-Repudiation

In a distributed, trustless environment lacking a central authority, relying on the assumption that data remains unaltered during transit is impossible. Within the **Design Science Research Process (DSRP)**, Module 5 dictates the security architecture of the artifact. It serves as the direct technical implementation of the Non-Repudiation of Origin (NRO) principles conceptually mandated by Module 1. This module governs the cryptographic security model of the *Finalized Provenance Components* (*FPCs*), ensuring the mathematical provability of data origin, creation time, and structural integrity.

3.6.1 Core Responsibilities and Managed Concepts

The primary responsibility of this module is to define how the Meta-Bundle (defined in Module 4) is cryptographically secured. It explicitly operationalizes the CPF's NRO policies (specifically NRO-P1 to NRO-P10) through the following concepts:

- **Verification Token (Proof Object):** The structural model of the digital signature applied to the FPC. Module 5 dictates that this token must encapsulate the signature payload, the hashing algorithm, and the identifier of the public key or certificate (satisfying NRO-P1 and NRO-P4).

- **Canonicalization and Signed Manifest:** The precise definition of *what* is being signed. Since the provenance graph exists as RDF, this module establishes the rules for deterministic normalization and serialization (e.g., RDF Dataset Canonicalization) prior to hashing. This ensures that functionally identical graphs produce identical hashes regardless of their internal node ordering.

- **Identity Binding:** The rules for cryptographically linking an external security credential (such as an X.509 certificate) to the abstract `cpm:SenderAgent` or `cpm:MainActivity` defined in the graph (Module 3).

- **Cryptographic Agility:** The framework's ability to support various cryptographic primitives (e.g., SHA-256, Ed25519, or post-quantum algorithms) and explicitly version them within the Meta-Bundle, ensuring resilience against cryptographic obsolescence.

3.6.2 Scope Boundaries (Out of Scope)

To prevent the CPF from overreaching into generalized IT security, Module 5 delineates strict boundaries:

1. **Key Management (PKI):** While Module 5 mandates the inclusion of a public key reference, it does not govern the processes of issuing, storing, or revoking private keys. That remains the responsibility of external Certificate Authorities (CA) or institutional Key Management Systems.

2. **Access Control (Authorization):** This module proves *who* performed an action and guarantees the data was not tampered with (Authentication/Integrity). It explicitly does *not* determine whether the agent *had the right* to perform the action (Role-Based Access Control). This logic is delegated to the domain-level application.

3. **Content Encryption:** The CPF focuses on transparency and auditability. If a payload (e.g., genomic data) must be encrypted for privacy, the CPF treats the encrypted file as an opaque binary blob and signs its hash. Decryption logic is out of scope.

3.6.3 Change Decision Matrix

Security rules are highly sensitive to change, as they directly impact the legal and technical auditability of the entire provenance chain:

- **Security Invariant Reduction (Major Change):** Removing mandatory attributes from the token (e.g., making the cryptographic timestamp optional) violates the core NRO-P3 principle established in Module 1, constituting a breaking Major change that would invalidate compliance.
- **Serialization Logic Alteration (Major Change):** Changing the canonicalization algorithm (e.g., moving from one RDF normalization standard to another) alters the resulting hashes. Without explicit migration protocols, verification of historical data will fail, categorizing this as a Major change.
- **Algorithm Addition (Minor Change):** Introducing support for a new signature algorithm extends the specification. As long as parsers maintain backward compatibility with previous algorithms, this represents a Minor change.

3.6.4 Boundary Resolution and Edge Cases

A critical edge case in long-term multi-organizational provenance is **Algorithmic Expiration**—for instance, if a critical collision vulnerability is discovered in the mandated SHA-256 algorithm five years after an FPC was published.

The CPF governance resolution dictates that Module 5 cannot simply delete the compromised algorithm from the validation specification, as this would render years of historical medical/scientific provenance unreadable. Instead, the module utilizes its *Cryptographic Agility* feature. The governance decision mandates a Minor version update to Module 5 that marks the old algorithm as `Deprecated` and enforces a new standard (e.g., SHA-3) for all newly generated FPCs.

Crucially, how is the old data handled? According to the interplay between Module 4 and Module 5, historical records signed with the deprecated algorithm remain immutable. They are structurally readable by parsers (Module 8) but must be flagged as cryptographically "Weak" or "Historical" during the validation phase. If an institution wishes to upgrade the security of an old record, they must issue a completely new FPC (v2) with a modern signature, explicitly utilizing the `cpm:referencedMetaBundleId` superseding mechanism defined in **Module 4**.

3.7 Module 6: CPM Traversal Logic

While Module 3 defines the static topology of the provenance graph, Module 6 governs its dynamic behavior. Within the **Design Science Research Process** (**DSRP**), this module operationalizes the artifact's functionality, establishing the abstract algorithmic logic required to navigate the distributed graph. Its primary goal is to ensure that any software implementation yields mathematically consistent traceability, resilience, and trust calculation results across the entire ecosystem, fulfilling the interoperability mandate (R4).

3.7.1 Core Responsibilities and Managed Concepts

The module standardizes the normative procedures for graph navigation and evaluation, decoupled from any specific programming language. It manages the following core concepts:

- **Directional Traceability Logic:** The recursive algorithmic steps for reconstructing lineage. This encompasses *Backward Traceability* (resolving `cpm:BackwardConnector` to find the origin) and *Forward Traceability* (resolving `cpm:ForwardConnector` for impact analysis).
- **Redundant Path Resolution:** A critical CPF governance rule. Module 6 dictates the exact algorithmic behavior for parsing *Redundant Connectors*. It defines how an algorithm must seamlessly "jump over" inaccessible nodes by utilizing redundant topological data to maintain an unbroken trace.
- **Trust and Integrity Evaluation:** The rules for aggregating the validity of individual nodes into a comprehensive trust metric. While Module 5 verifies the signature, Module 6 governs the *Hash Chain Validation*—the algorithm must strictly compare the `cpm:hashValue` stored in the local connector (Module 2) against the dynamically calculated hash of the remotely fetched Meta-Bundle (Module 4) to detect silent tampering.
- **Subgraph Expansion Rules:** The logic governing when a traversal algorithm should treat a `cpm:MainActivity` as an opaque

"black box" versus when it should fetch the detailed internal provenance bundle it references.

3.7.2 Scope Boundaries (Out of Scope)

To preserve strict Separation of Concerns, the Traversal Logic explicitly excludes several operational domains:

- **Data Structures:** Module 6 dictates *how* to use forward and backward connectors for navigation, but the validation of their existence, constraints, and cardinalities belongs strictly to **Module 3 (Backbone)** and **Module 9 (Ontology)**.
- **Cryptographic Mathematics:** The actual mathematical verification of a digital signature is the domain of **Module 5 (Security)**. Module 6 merely consumes the Boolean result (Valid/Invalid) from Module 5 to determine the subsequent traversal path.
- **Performance Optimization:** Implementation details such as graph caching, database indexing, or concurrent asynchronous fetching are out of scope. They are operational concerns managed by **Module 8 (Reference Software)**.

3.7.3 Change Decision Matrix

Changes to the traversal logic directly impact the outcome of provenance queries, legal audits, and security evaluations. They are evaluated as follows:

- **Trust Calculation Alteration (Major Change):** Proposing a rule change that introduces "tolerance" for broken hash chains (e.g., allowing one missing link to pass without flagging the subsequent chain as untrusted) fundamentally alters the NRO audit results. This is a critical Major change.
- **Algorithmic Filtering (Major Change):** Modifying the base algorithm to automatically omit certain classes of agents (e.g., ignoring `cpm:ProviderAgent` nodes) redefines what constitutes a "complete history," requiring a Major version update.

- **Output Format Modification (Rejection):** A proposal to standardize the algorithmic output format (e.g., demanding the algorithm returns JSON-LD instead of an abstract Node List) is rejected from Module 6. Module 6 defines the *logical content* of the result; the formatting belongs to the API interfaces in **Module 8**.

3.7.4 Boundary Resolution and Edge Cases

A paramount edge case in distributed ecosystems is **The Unreachable Node (Dead Link)**. For example, an auditor traces a blood sample from a Biobank back to Hospital A, but Hospital A's provenance server is permanently offline (HTTP 404). A naive traversal algorithm would crash or return an incomplete graph, violating the Resilience requirement (R5).

The CPF governance framework resolves this explicitly within Module 6. The governance decision mandates that the traversal algorithm must implement a *Redundant Fallback Protocol*. Upon encountering a dead link via a primary `cpm:BackwardConnector`, the algorithm is strictly required to scan the current FPC for a corresponding *Redundant Connector*. The logic must then evaluate this redundant link to bypass the offline Hospital A, connecting directly to the preceding entity (e.g., the local clinic that sent the sample to Hospital A), while flagging the "jump" in the final audit report. Any software implementation failing to apply this fallback logic is deemed non-compliant with the CPF specification.

3.8 Module 7: CPM Ontology & Validation Shapes

While Modules 2, 3, and 5 establish the conceptual rules and structures of the framework (defining what entities are and how they behave), Module 7 provides their formal, machine-readable representation. Within the **Design Science Research Process (DSRP)**, this module represents the core instantiation of the technical artifact. To enable automated processing and ensure semantic interoperability across disparate software systems, the theoretical constraints are translated into strict, standardized semantic data formats.

3.8.1 Core Responsibilities and Managed Concepts

This module manages the formal vocabulary and the validation constraints of the Common Provenance Framework. It bridges the gap between abstract architecture and implementation by governing the following artifacts:

- **The Semantic Vocabulary (`cpm.ttl`):** The formal definition of classes, attributes, and relations in OWL 2. This includes defining exact Internationalized Resource Identifiers (IRIs) under the `cpm:` namespace, data types, and structural domain/range boundaries. Crucially, it establishes the normative semantic mapping to overarching standards, such as asserting `subClassOf` relationships to the W3C PROV-O ontology.
- **Executable Validation Shapes (`cpm-shapes.ttl`):** A set of strict technical rules implemented via SHACL (Shapes Constraint Language). This component acts as the automated "compliance officer" of the framework. It actively verifies whether a given RDF dataset conforms to the topological rules of Module 3 and the ID structures of Module 2 (e.g., executing the rule that a `cpm:ReceiverAgent` MUST contain a valid `cpm:externalIdType`).
- **Namespace and Prefix Governance:** The management of the official `cpm:` prefix and the versioning of the ontology files themselves, ensuring that parsers can explicitly identify and resolve the vocabulary version being utilized.

3.8.2 Scope Boundaries (Out of Scope)

Module 7 enforces rules but does not invent them. The Separation of Concerns dictates:

- **Defining the "Law":** The conceptual mandate that an *Activity* must be linked to an *Agent* is defined by **Module 3 (Backbone)**. Module 7 merely translates this law into a SHACL shape (`sh:property`) to execute the validation.
- **Algorithmic Execution:** How the provenance graph is recursively traversed is the domain of **Module 6 (Traversal Logic)**.

Module 7 only defines the static topological structures that the algorithm reads.

- **Data Storage:** Whether the ontology is deployed in a Graph Triplestore, a relational database, or stored as flat files is an implementation detail deferred to the **Reference Software**.

3.8.3 Change Decision Matrix

Modifications to the ontology and validation shapes directly impact data ingestion pipelines and compliance audits. They are evaluated as follows:

- **Semantic Re-identification (Major Change):** Renaming a core IRI (e.g., changing `cpm:hashValue` to `cpm:checksum`) is a breaking Major change, as historical FPCs utilizing the previous IRI will fail SHACL validation.
- **Constraint Tightening (Major Change):** Adding a new SHACL rule that begins rejecting previously valid data (e.g., changing `sh:minCount 0` to `sh:minCount 1` for an attribute) constitutes a Major change.
- **Constraint Relaxation (Minor Change):** Modifying a SHACL shape to allow newly added optional structures without breaking the validation of old records is a Minor change.
- **Documentation Addition (Patch):** Adding text annotations (e.g., `rdfs:comment` or `skos:definition`) to an existing class for human readability does not alter machine validation logic and represents a Patch.

3.8.4 Boundary Resolution and Edge Cases

A highly technical edge case governed by this module is the **Domain Extension vs. Graph Integrity** dilemma. A hospital might wish to inject specialized medical diagnostics (e.g., SNOMED CT terms) directly into the `cpm:MainActivity` node. A completely "Open" validation framework would allow this, but it risks structural bloat and

canonicalization failures (Module 5) if arbitrary external triples are haphazardly injected.

The CPF governance framework resolves this by enforcing a strictly controlled architecture in **Module 7**. The resolution dictates the use of the **Closed-World Assumption** in SHACL validation. As instantiated in the `cpm-shapes.ttl` artifact, core shapes (such as `cpm:ReceiverAgentShape`) are explicitly defined with `sh:closed true`. Any unregistered external property automatically triggers a validation failure. To permit safe, compliant domain extensions, Module 7 mandates the explicit use of `sh:ignoredProperties` for whitelisted attributes, or the designation of specific extension nodes. This ensures that the core provenance backbone remains pure, highly predictable, and cryptographically stable, while still allowing hospitals to append necessary clinical metadata.

3.9 Module 8: Reference Software & Tooling

Theoretical specifications (Modules 0–6) and formal ontologies (Module 7) are insufficient to drive practical adoption without verified software tooling. Within the **Design Science Research Process (DSRP)**, Module 8 represents the *Design and Development* and *Demonstration* phases. It manages the canonical software implementation of the Common Provenance Framework. It serves a dual purpose: acting as a functional Proof of Concept for the theoretical rules and providing a "gold standard" against which all third-party software implementations are evaluated.

Crucially, this module absorbs the practical implementation of network exchanges. By intentionally omitting a theoretical "Exchange Protocols" module to preserve technological agnosticism (satisfying the R1 constraint in Module 1), the CPF delegates the actual execution of data transfer (e.g., REST API or gRPC client logic) to the Reference Software libraries managed here.

3.9.1 Core Responsibilities and Managed Concepts

The primary responsibility of this module is to provide developers with reliable foundational tools and to act as the ultimate compliance arbitrator.

The module manages the source codes and documentation for the following components:

- **Generator Libraries (SDKs):** Software libraries that abstract the complexity of RDF generation. They allow external developers to programmatically create valid topological graphs (Module 3), hash data (Module 5), and package them into RO-Crate Meta-Bundles (Module 4) without needing to manually write semantic triples.
- **Validation Engine:** An automated service that ingests external provenance bundles and verifies them against the `cpm-shapes.ttl` SHACL constraints defined in Module 7.
- **Traversal & Transport Execution:** The concrete software implementation of the abstract traversal algorithms defined in Module 6. This includes the functional network interfaces (HTTP clients, parsers) required to fetch remote Meta-Bundles based on URI linkage.
- **Conformance Test Suite:** A comprehensive set of test data, including both positive workflows and negative (tampered or topologically invalid) scenarios, used to certify whether third-party implementations are strictly CPF-compliant.

3.9.2 Scope Boundaries (Out of Scope)

A strict boundary distinguishes reference libraries from end-user production systems:

- **Production Domain Systems:** Module 8 does not govern or dictate the development of specific clinical or laboratory software (such as Laboratory Information Management Systems - LIMS). It strictly provides the agnostic middleware libraries those systems might utilize.
- **Theoretical Supremacy:** Software does not dictate the rules. If the reference implementation behaves contrary to the structural laws defined in Module 3 (Backbone) or the ontology in Module 7, this is classified as a software bug, not a change in

the governance rules. The text specification *always* supersedes the code implementation.

3.9.3 Change Decision Matrix

Software maintenance has its own lifecycle, but in a governance framework, it is heavily dictated by downward propagation:

- **Downward Propagation (Major/Minor Update):** If a higher-level conceptual module changes (e.g., Module 2 introduces a new standard for `cpm:externalIdType`), Module 8 is forced to update its parsers and generator libraries to support it. This demonstrates the cascading nature of the Change Propagation Matrix.
- **Bug Fixes (Patch):** Correcting a memory leak or a crash during the loading of large RO-Crates does not alter the framework's theoretical behavior and is released as a Patch.
- **Specification Discrepancy (Incident):** If the Validation Engine rejects a graph that the text of Module 3 claims should be valid, an incident is declared. The governance body must decide whether the text is ambiguous (requiring a Module 3 update) or the code is flawed (requiring a Module 8 patch).

3.9.4 Boundary Resolution and Edge Cases

A crucial edge case arises regarding software support for deprecated security features. For example, if Module 5 declares the SHA-256 algorithm "weak" in the future and mandates SHA-3, the software engineers cannot simply delete the SHA-256 validation logic from the Reference Implementation in the next update.

The CPF governance resolution mandates **Asymmetric Backward Compatibility**. To ensure that historical, cryptographically sealed provenance graphs remain readable, Module 8 must retain the parsing and validation logic for the deprecated algorithm. However, the Generator Libraries (SDKs) must be strictly updated to disable or throw exceptions if a system attempts to create *new* provenance data using the deprecated feature. This governance rule ensures the ecosystem

evolves securely without severing its connection to historical scientific records.

3.10 Governance Dynamics: The Change Propagation Matrix

The formal definition of isolated modules (Modules 0 through 8) is a necessary prerequisite for a governance framework, but it is insufficient for long-term maintenance. In a highly interdependent distributed ecosystem like the Common Provenance Framework, an isolated modification to one component frequently triggers a cascading failure in another. Within the **Design Science Research Process** (**DSRP**), understanding and managing these cascading effects is critical for ensuring the *Robustness of the Artifact*.

To operationalize the governance of future CPF updates, this thesis introduces the **Change Propagation Matrix** (Table 3.1). This matrix acts as the central diagnostic tool for the governance body, explicitly formalizing the unidirectional dependencies between architectural layers.

3.10.1 Matrix Architecture and Reading Logic

The matrix maps the origin of a change against its systemic impact.

- **Columns (Trigger $\rightarrow$):** Represent the module where a change is officially proposed and approved (e.g., redefining a cryptographic requirement).
- **Rows (Affected Module $\downarrow$):** Represent the downstream modules that must be immediately audited, and potentially updated, to maintain framework compliance. An 'x' denotes a mandatory compliance check.

3.10.2 Analytical Dynamics and Cascading Effects

As illustrated in Table 3.1, the matrix reveals several critical architectural invariants of the CPF governance model. The dependencies

Table 3.1: Change Propagation Matrix: Inter-module Dependencies within the CPF Governance Framework

Affected Module ↓ / Trigger →	M0	M1	M2	M3	M4	M5	M6	M7	M8
M0: Terminology	–								
M1: Architectural Constraints	x	–							
M2: Identity & Linkage	x	x	–						
M3: Traversal Backbone	x	x	x	–					
M4: Meta-Bundles & Versioning	x	x	x	x	–				
M5: Security & NRO	x	x	x	x		–			
M6: Traversal Logic		x	x	x	x		–		
M7: Ontology & Validation	x	x	x	x	x	x		–	
M8: Reference Software	x	x	x	x	x	x	x	x	–

within the framework exhibit specific behavioral patterns that dictate the governance process.

The Principle of Downward Propagation. Dependencies are strictly unidirectional, flowing from abstract concepts down to technical implementations. As evidenced by the matrix, **Module 8** (**Reference Software**) depends on every prior module. Conversely, no higher-level module depends on Module 8 or Module 7. This formally encodes the governance rule established in Section 3.9: A software bug or an ontological parsing error cannot dictate a change in the topological laws of the framework.

Topological Ripple Effect. The matrix highlights cascading impacts, particularly from core structural changes. For example, if a change is introduced in **Module 2** (**Identity & Linkage**)—such as mandating that the `cpm:externalIdType` attribute supports a new URI format—the matrix dictates a cascade. The ontology (**Module 7**) must be updated to adjust the `cpm-shapes.ttl` SHACL constraints. Subsequently, the Reference Software (**Module 8**) must update its parsers to avoid false-positive validation failures.

Algorithmic Independence. The matrix explicitly shows no dependency between **Module 5** (**Security**) and **Module 6** (**Traversal Logic**). This highlights a highly optimized Separation of Concerns. If the governance body deprecates a hashing algorithm in Module 5, the traversal algorithm in Module 6 does not need to be rewritten. Module 6 merely consumes the Boolean output (Valid/Invalid) provided by

the security layer, ensuring that cryptographic agility does not break graph navigation logic.

The Meta-Bundle Nexus. Finally, changes to **Module 4 (Meta-Bundles)** uniquely impact both the static validation shapes (Module 7) and the dynamic traversal logic (Module 6). Because Module 4 governs how a disconnected graph references external payloads (Detached Architecture), any change to its structure alters the functional pathways the algorithm must navigate to reconstruct a complete history.

3.10.3 Architectural Synthesis: The Layered Governance Stack

While the Change Propagation Matrix (Table 3.1) provides a rigid diagnostic tool for dependency tracking, comprehending the Common Provenance Framework requires a holistic cognitive model. To prevent the architectural visualization from degrading into an unintelligible web of intersecting dependencies (a "spaghetti diagram"), this thesis abstracts the 9 isolated modules into a **Layered Governance Stack** (Figure 3.1).

This visualization aligns with standard IT architectural models (such as the OSI model or HL7 FHIR structures) by grouping the modules into four sequential tiers based on their level of abstraction:

1. **Conceptual Foundation (Layer 1):** The absolute base of the framework. Terminology and Architectural Constraints define the "constitution" of the CPF. They dictate the rules but contain no technical specifications.

2. **Topological & Data Structures (Layer 2):** The structural tier. These modules define the shape of the graph, how identities are formed, and how data is packaged into immutable bundles.

3. **Operational Logic & Rules (Layer 3):** The execution tier. Here, static data is evaluated for security, semantic validity, and algorithmic traversability.

4. **Implementation (Layer 4):** The pinnacle of the stack. This layer represents the physical Reference Software that instantiates all underlying theoretical layers into executable code.

As depicted by the propagation arrow in Figure 3.1, governance changes ripple upward through the stack. A modification in the conceptual foundation inherently forces adaptation in the structural, logical, and ultimately, the implementation layers.

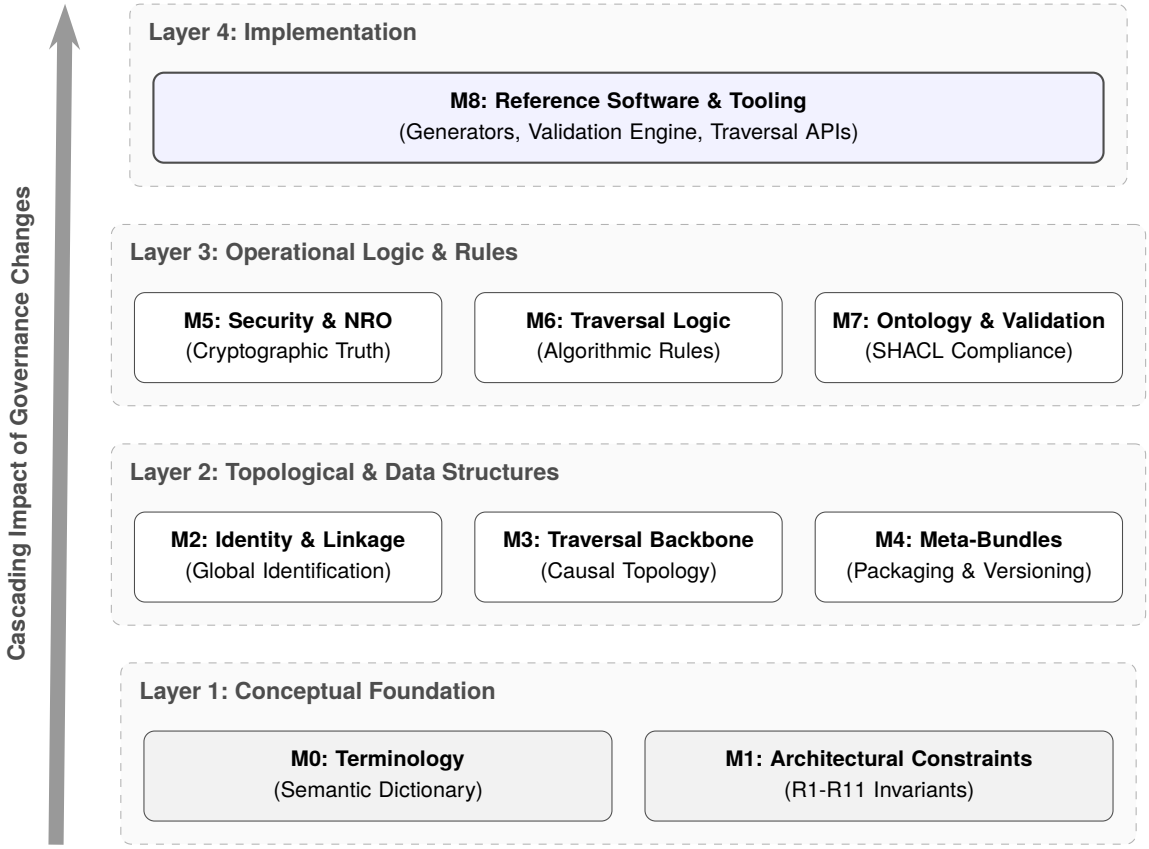

Figure 3.1: The Layered Governance Stack of the Common Provenance Framework.

4 Technical Formalization of the Framework

While Chapter 3 established the abstract rules and governance policies, this chapter transitions into the practical instantiation of the framework. It details the technical artifacts developed to enforce the governance model.

4.1 The CPM Ontology: Details the hierarchical design of the core classes (Agent, Entity, Activity) implemented in OWL 2, explicitly demonstrating the semantic mapping and extension of the underlying W3C PROV-O concepts to fit the CPF multi-organizational topology.

4.2 Automated Validation with SHACL: Presents the executable validation constraints (Shapes). It explains the technical mechanisms used to automatically enforce structural invariants, such as mandating the cryptographic linkage between Activities and responsible Agents.

5 Demonstration: Multi-Organizational Provenance Scenario

Aligning with the Demonstration phase of the Design Science Research Process, this chapter validates the proposed framework and its formal ontology through a practical, multi-organizational case study.

5.1 Scenario Overview: Introduces a realistic life-sciences workflow tracking a biological sample as it traverses three independent administrative domains: a Primary Hospital, a Biobank, and a Research Laboratory.

5.2 Data Generation and Distribution: Demonstrates the instantiation of the ontology in practice, illustrating the structural composition of the individual Meta-Bundles generated at each node of the supply chain.

5.3 Cross-Bundle Tracing: Analyzes the execution of the Traversal Logic (Module 6), demonstrating how the algorithm reconstructs the complete historical lineage by seamlessly navigating cryptographic links across disconnected organizational bundles.

6 Evaluation and Discussion

This chapter critically evaluates the designed artifacts and the governance framework against the baseline architectural constraints established in Chapter 3.

6.1 Requirements Fulfillment: Provides a systematic comparative analysis (Traceability Matrix), objectively assessing how the technical implementation satisfies the core CPF constraints (e.g., decentralization, Non-Repudiation of Origin, and domain agnosticism).

6.2 Security and Trust Analysis: Discusses the resilience of the framework against potential operational failures and malicious tampering, detailing how the model handles broken cryptographic signatures and disconnected chains.

6.3 Scalability and Performance Considerations: Explores the behavior of the framework under heavy data loads. It evaluates the efficiency of the "Detached Payload" architecture (referencing external big data) and its capability to scale in massive distributed ecosystems without compromising cryptographic integrity.

7 Conclusion

Summary of the work...

A SHACL Profiles Source Code